

Université Sorbonne Paris Nord
Ecole d'Ingénieurs Sup Galilée

TP4

Réalisé par:

BRAHIMI Kaoutar

Exercice n°1

1- Addition de deux entiers :

```
DECLARE

a NUMBER;

b NUMBER;

BEGIN

a := TO_NUMBER('&Entrez_le_premier_nombre');

b := TO_NUMBER('&Entrez_le_deuxieme_nombre');

DBMS_OUTPUT.PUT_LINE('La somme de ' || a || ' et ' || b || ' est : ' || (a + b));

END;

/
```

2. Table de multiplication :

```
DECLARE

n NUMBER;

i NUMBER := 1;

BEGIN

n := TO_NUMBER('&Entrez_un_nombre');

DBMS_OUTPUT.PUT_LINE('Table de multiplication de ' || n || ':');

WHILE i <= 10 LOOP

DBMS_OUTPUT.PUT_LINE(n || ' x ' || i || ' = ' || (n * i));

i := i + 1;

END LOOP;

END;

/
```

3- Fonction récursive : puissance :

```
CREATE OR REPLACE FUNCTION puissance(x IN NUMBER, n IN NUMBER) RETURN
NUMBER IS
```

```

BEGIN
  IF n < 0 THEN
    RETURN NULL;
  ELSIF n = 0 THEN
    RETURN 1;
  ELSE
    RETURN x * puissance(x, n - 1);
  END IF;
END;
/

```

4- Factorielle avec insertion en base :

```

CREATE TABLE resultatFactoriel (
  nombre NUMBER,
  factoriel NUMBER
);

```

```

DECLARE
  n NUMBER;
  fact NUMBER := 1;
  i NUMBER := 1;
BEGIN
  n := TO_NUMBER('&Entrez_un_nombre_strictement_positif');
  IF n <= 0 THEN
    DBMS_OUTPUT.PUT_LINE('Le nombre doit être strictement positif');
  ELSE
    FOR i IN 1..n LOOP
      fact := fact * i;
    END LOOP;
  END IF;
END;

```

```

INSERT INTO resultatFactoriel (nombre, factoriel) VALUES (n, fact);

DBMS_OUTPUT.PUT_LINE('La factorielle de ' || n || ' est : ' || fact);

END IF;

END;

/

```

5. Factorielles des 20 premiers nombres :

```

CREATE TABLE resultatsFactoriels (

nombre NUMBER,

factoriel NUMBER

);


DECLARE

fact NUMBER := 1;

BEGIN

FOR i IN 1..20 LOOP

fact := 1;

FOR j IN 1..i LOOP

fact := fact * j;

END LOOP;

INSERT INTO resultatsFactoriels (nombre, factoriel) VALUES (i, fact);

END LOOP;

DBMS_OUTPUT.PUT_LINE('Les factorielles de 1 à 20 ont été calculées et stockées.');
```

END;

/

Exercice n°2

1- Insertion d'un nouvel employé :

```
DECLARE
v_employe emp%ROWTYPE;
BEGIN
v_employe.matr := 4;
v_employe.nom := 'Youcef';
v_employe.sal := 2500;
v_employe.adresse := 'avenue de la République';
v_employe.dep := 92002;
INSERT INTO emp VALUES v_employe;
DBMS_OUTPUT.PUT_LINE('Employé inséré avec succès.');
```

END;

/

2- Suppression de clients par département :

```
DECLARE
v_nb_lignes NUMBER;
BEGIN
DELETE FROM emp WHERE dep = 10;
v_nb_lignes := SQL%ROWCOUNT;
DBMS_OUTPUT.PUT_LINE('Nombre d"employés supprimés : ' || v_nb_lignes);
END;
```

/

3- Somme des salaires avec curseur et boucle :

```
SET SERVEROUTPUT ON;
DECLARE
v_salaire emp.sal%TYPE;
```

```

v_total emp.sal%TYPE := 0;

CURSOR c_salaires IS SELECT sal FROM emp;

BEGIN

OPEN c_salaires;

LOOP

FETCH c_salaires INTO v_salaire;

EXIT WHEN c_salaires%NOTFOUND;

IF v_salaire IS NOT NULL THEN

v_total := v_total + v_salaire;

END IF;

END LOOP;

CLOSE c_salaires;

DBMS_OUTPUT.PUT_LINE('Somme des salaires : ' || v_total);

END;

/

```

4- Calcul du salaire moyen avec boucle LOOP :

```

SET SERVEROUTPUT ON;

DECLARE

v_salaire emp.sal%TYPE;

v_total emp.sal%TYPE := 0;

v_count NUMBER := 0;

v_moyenne NUMBER;

CURSOR c_salaires IS SELECT sal FROM emp;

BEGIN

OPEN c_salaires;

LOOP

FETCH c_salaires INTO v_salaire;

EXIT WHEN c_salaires%NOTFOUND;

```

```

        IF v_salaire IS NOT NULL THEN

            v_total := v_total + v_salaire;

            v_count := v_count + 1;

        END IF;

    END LOOP;

    CLOSE c_salaires;

    IF v_count > 0 THEN

        v_moyenne := v_total / v_count;

        DBMS_OUTPUT.PUT_LINE('Moyenne des salaires : ' || v_moyenne);

    ELSE

        DBMS_OUTPUT.PUT_LINE('Aucun salaire à traiter.');

```

5- Versions avec la boucle FOR...IN

Somme des salaires :

```

SET SERVEROUTPUT ON;

DECLARE

    v_total emp.sal%TYPE := 0;

    CURSOR c_salaires IS SELECT sal FROM emp;

BEGIN

    FOR ligne IN c_salaires LOOP

        IF ligne.sal IS NOT NULL THEN

            v_total := v_total + ligne.sal;

        END IF;

    END LOOP;


```

```

DBMS_OUTPUT.PUT_LINE('Somme des salaires (FOR) : ' || v_total);

END;

/

```

Moyenne des salaires :

```

SET SERVEROUTPUT ON;

DECLARE

v_total emp.sal%TYPE := 0;

v_count NUMBER := 0;

v_moyenne NUMBER;

CURSOR c_salaires IS SELECT sal FROM emp;

BEGIN

FOR ligne IN c_salaires LOOP

IF ligne.sal IS NOT NULL THEN

v_total := v_total + ligne.sal;

v_count := v_count + 1;

END IF;

END LOOP;


IF v_count > 0 THEN

v_moyenne := v_total / v_count;

DBMS_OUTPUT.PUT_LINE('Moyenne des salaires (FOR) : ' || v_moyenne);

ELSE

DBMS_OUTPUT.PUT_LINE('Aucun salaire à traiter.');
```

```

END IF;

END;

/

```

6- Procédure anonyme avec curseur paramétré :

```

SET SERVEROUTPUT ON;

DECLARE

CURSOR c(p_dep emp.dep%TYPE) IS

    SELECT dep, nom FROM emp WHERE dep = p_dep;

BEGIN

    -- Parcours pour le département 92000

    FOR v_employe IN c(92000) LOOP

        DBMS_OUTPUT.PUT_LINE('Dep 92000 : ' || v_employe.nom);

    END LOOP;


    -- Parcours pour le département 75000

    FOR v_employe IN c(75000) LOOP

        DBMS_OUTPUT.PUT_LINE('Dep 75000 : ' || v_employe.nom);

    END LOOP;

END;

/

```

Exercice n°3

Définition du package :

```

CREATE OR REPLACE PACKAGE gestion_clients AS

    PROCEDURE ajouter_client(p_id_client NUMBER, p_nom VARCHAR2, p_adresse VARCHAR2);

    PROCEDURE ajouter_client(p_nom VARCHAR2, p_adresse VARCHAR2);

END gestion_clients;

/

```

Corps du package :

```

CREATE OR REPLACE PACKAGE BODY gestion_clients AS

```

```

PROCEDURE ajouter_client(p_id_client NUMBER, p_nom VARCHAR2, p_adresse
VARCHAR2) IS

    v_count NUMBER;

BEGIN

    -- Vérifier si l'ID client existe déjà

    SELECT COUNT(*) INTO v_count FROM clients WHERE id_client = p_id_client;

    IF v_count > 0 THEN

        RAISE_APPLICATION_ERROR(-20001, 'ID client déjà utilisé. Veuillez spécifier un
autre ID.');
```

```

    END IF;

    -- Insérer le nouveau client

    INSERT INTO clients (id_client, nom, adresse)

    VALUES (p_id_client, p_nom, p_adresse);

    -- Message et commit

    DBMS_OUTPUT.PUT_LINE('Client ajouté avec succès avec ID : ' || p_id_client);

    COMMIT;

EXCEPTION

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Erreur lors de l'ajout du client : ' || SQLERRM);

        ROLLBACK;

    END ajouter_client;

PROCEDURE ajouter_client(p_nom VARCHAR2, p_adresse VARCHAR2) IS

    v_id_client NUMBER;

BEGIN

    -- Générer un nouvel ID via une séquence
```

```
SELECT seq_client.NEXTVAL INTO v_id_client FROM DUAL;
```

```
-- Insérer le client
```

```
INSERT INTO clients (id_client, nom, adresse)
```

```
VALUES (v_id_client, p_nom, p_adresse);
```

```
-- Message et commit
```

```
DBMS_OUTPUT.PUT_LINE('Client ajouté avec succès avec ID généré automatiquement : ' || v_id_client);
```

```
COMMIT;
```

```
EXCEPTION
```

```
WHEN OTHERS THEN
```

```
DBMS_OUTPUT.PUT_LINE('Erreur lors de l'ajout du client : ' || SQLERRM);
```

```
ROLLBACK;
```

```
END ajouter_client;
```

```
END gestion_clients;
```

```
/
```